

Chuyên đề: Quy hoạch Động Cơ bản (Basic Dynamic Programming)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

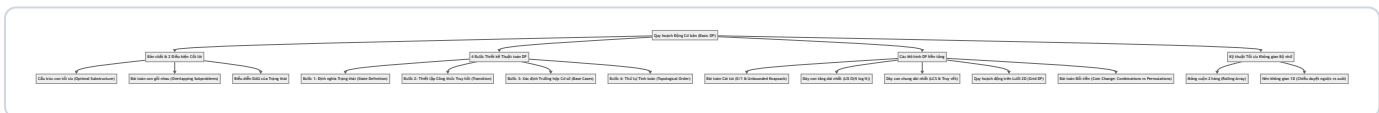
Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Tổng quan & Bản đồ Tư duy (Mindmap)

Quy hoạch động (Dynamic Programming - DP) là phương pháp thiết kế giải thuật quan trọng bậc nhất trong Lập trình thi đấu (ICPC, Olympic Tin học, Codeforces). Bản chất của Quy hoạch động là: **"Giải quyết một bài toán phức tạp bằng cách phân rã nó thành các bài toán con gối nhau, giải quyết từng bài toán con đúng một lần và lưu trữ lời giải (bằng mảng hoặc bảng nhớ) để tái sử dụng, tránh việc tính toán lặp lại theo hàm mũ."**

Về mặt hình học và lý thuyết đồ thị, mọi thuật toán Quy hoạch động đều tương đương với việc tìm đường đi tối ưu (hoặc đếm số đường đi) trên một **Đồ thị có hướng không chu trình (Directed Acyclic Graph - DAG)** của các trạng thái.



2. Cơ sở Lý thuyết & 4 Bước Thiết kế Thuật toán DP Chuẩn mực

2.1. Hai Điều kiện Tiên quyết của Quy hoạch Động

Một bài toán có thể giải quyết bằng Quy hoạch động khi và chỉ khi thỏa mãn hai tính chất:

- Cấu trúc con tối ưu (Optimal Substructure):** Lời giải tối ưu của bài toán lớn được xây dựng trực tiếp từ lời giải tối ưu của các bài toán con cấp thấp hơn.
- Các bài toán con gối nhau (Overlapping Subproblems):** Cùng một bài toán con được gọi đi gọi lại rất nhiều lần trong cây đệ quy. Nếu không lưu lại (Memoization/Tabulation), độ phức tạp sẽ bùng nổ theo hàm mũ $O(2^N)$ hoặc $O(N!)$.

2.2. Hai Phương pháp Cài đặt DP: Top-Down vs. Bottom-Up

- Đệ quy có nhớ (Top-Down with Memoization):**
- Bắt đầu từ bài toán lớn cần tính, gọi đệ quy xuống các bài toán con.

- Sử dụng mảng nhớ `memo[...]` để lưu kết quả ngay khi tính xong. Nếu trạng thái đã được tính trước đó, trả về giá trị ngay trong $O(1)$.
- *Ưu điểm*: Tự nhiên, chỉ tính những trạng thái thực sự cần thiết (unvisited states skipped).
- *Nhược điểm*: Tốn chi phí gọi hàm (Call Stack Overhead), có nguy cơ tràn bộ nhớ ngăn xếp (Stack Overflow).
- **Tính toán từ dưới lên (Bottom-Up with Tabulation)**:
- Khởi tạo các trường hợp cơ sở (Base Cases).
- Sử dụng các vòng lặp `for` để tính toán tuần tự các trạng thái theo thứ tự tô-pô (Topological Sort).
- *Ưu điểm*: Tốc độ chạy cực nhanh, không tốn ngăn xếp đệ quy, dễ dàng áp dụng kỹ thuật **tối ưu hóa không gian bộ nhớ (Space Optimization)**.

2.3. Quy trình 4 Bước Thiết kế Trạng thái DP

Khi đối mặt với một bài toán DP trong phòng thi, hãy luôn tuân thủ 4 bước:

1. Định nghĩa Trạng thái (State Definition):

- Trả lời câu hỏi: "Các tham số nào là tối thiểu để đại diện cho một cấu hình độc lập của bài toán con?"
- Ví dụ: $dp[i][w]$ là giá trị lớn nhất khi chỉ xét i đồ vật đầu tiên với tổng khối lượng không vượt quá w .

2. Thiết lập Công thức Truy hồi (State Transition Formula):

- Trả lời câu hỏi: "Trạng thái hiện tại được suy ra từ các trạng thái nào trước đó (hoặc đẩy giá trị đến các trạng thái nào tiếp theo)?"

3. Xác định Trường hợp Cơ sở (Base Cases):

- Xác định giá trị xuất phát của bài toán nhỏ nhất không thể phân rã thêm (ví dụ: $dp[0] = 0, dp[start] = 1$).
- Khởi tạo các giá trị còn lại bằng $-\infty$ (với bài toán Max) hoặc $+\infty$ (với bài toán Min).

4. Thứ tự Tính toán (Order of Computation):

- Đảm bảo rằng khi tính $dp[state]$, tất cả các trạng thái con $dp[substate]$ cần thiết đều đã được tính toán xong.

⚡ 3. Các Mô hình Quy hoạch Động Nền tảng (Classic DP Paradigms)

3.1. Mô hình Bài toán Cái túi (Knapsack Problem)

A. Cái túi 0/1 (0/1 Knapsack)

Cho N đồ vật. Đồ vật thứ i có khối lượng w_i và giá trị v_i . Mỗi đồ vật chỉ được chọn **tối đa 1 lần**. Tìm giá trị lớn nhất có thể thu được với sức chứa balo tối đa W .

- **Định nghĩa 2D**: $dp[i][w]$ là giá trị lớn nhất khi xét i vật đầu tiên, khối lượng $\leq w$.
- **Công thức truy hồi**:

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{nếu } w < w_i \\ \max(dp[i-1][w], dp[i-1][w - w_i] + v_i) & \text{nếu } w \geq w_i \end{cases}$$

- **Kỹ thuật nén bộ nhớ xuống mảng 1D:**

Nhận xét: $dp[i][w]$ chỉ phụ thuộc vào hàng trước đó $dp[i - 1][\dots]$. Ta có thể dùng một mảng 1D `dp[w]`.

📌 **Quan trọng:**

Quy tắc vàng 0/1 Knapsack: Bắt buộc phải duyệt biến khối lượng w **NGƯỢC TỪ W VỀ w_i !**

Vì nếu duyệt xuôi, giá trị $dp[w - w_i]$ đã được cập nhật bởi đồ vật i ở bước trước đó, dẫn đến việc đồ vật i bị chọn nhiều lần!

```
// 0/1 Knapsack tối ưu bộ nhớ O(W)
long long knapsack01(int W, const vector<int>& weights, const vector<long long>& values) {
    int n = weights.size();
    vector<long long> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        int w = weights[i];
        long long v = values[i];
        // Duyệt NGƯỢC từ W về w
        for (int j = W; j >= w; --j) {
            dp[j] = max(dp[j], dp[j - w] + v);
        }
    }
    return dp[W];
}
```

B. Cái túi Không giới hạn (Unbounded Knapsack)

Mỗi đồ vật được phép chọn **không giới hạn số lần**.

- **Công thức truy hồi 1D:**

$$dp[w] = \max(dp[w], dp[w - w_i] + v_i)$$

- **Quy tắc duyệt:** Duyệt biến khối lượng w **XUÔI TỪ w_i ĐẾN W** .

Việc duyệt xuôi cho phép trạng thái $dp[w]$ tái sử dụng kết quả của chính đồ vật đó tại $dp[w - w_i]$ (tương ứng với việc chọn đồ vật thêm lần thứ 2, thứ 3...).

3.2. Mô hình Dãy con Tăng Dài nhất (LIS - Longest Increasing Subsequence)

Cho dãy N số nguyên $a_1, a_2, \dots, a_N$. Tìm độ dài lớn nhất của một dãy con tăng nghiêm ngặt: $a_{i_1} < a_{i_2} < \dots < a_{i_k}$ với $i_1 < i_2 < \dots < i_k$.

A. Thuật toán DP $O(N^2)$:

- $dp[i]$: Độ dài LIS kết thúc tại phần tử a_i .
- Công thức: $dp[i] = 1 + \max_{j < i, a_j < a_i} dp[j]$.

B. Thuật toán Tối ưu $O(N \log N)$ (Patience Sorting + Binary Search):

Duy trì một mảng `tails`, trong đó `tails[len]` lưu **giá trị nhỏ nhất của phần tử kết thúc một dãy con tăng có độ dài len**.

- Mảng `tails` luôn là một dãy **tăng nghiêm ngặt**.
- Với mỗi phần tử x trong mảng ban đầu:
 - Dùng tìm kiếm nhị phân (`std::lower_bound`) để tìm vị trí đầu tiên trong `tails` có giá trị $\geq x$.
 - Nếu tìm thấy tại vị trí pos : Ta cập nhật `tails[pos] = x` (làm giảm giá trị kết thúc, tạo cơ hội tốt hơn cho các phần tử sau).
 - Nếu không tìm thấy (x lớn hơn tất cả phần tử trong `tails`): Thêm x vào cuối mảng `tails` (`tails.push_back(x)`), độ dài LIS tăng thêm 1.
- **Độ phức tạp**: $O(N \log N)$ thời gian, $O(N)$ bộ nhớ.

```
int lengthOfLIS(const vector<long long>& a) {
    vector<long long> tails;
    for (long long x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) {
            tails.push_back(x);
        } else {
            *it = x;
        }
    }
    return tails.size();
}
```

3.3. Mô hình Dãy con Chung Dài nhất (LCS - Longest Common Subsequence)

Cho hai chuỗi ký tự S độ dài N và T độ dài M . Tìm độ dài của chuỗi con chung (không nhất thiết liên tiếp) dài nhất của S và T .

- **Định nghĩa trạng thái**: $dp[i][j]$ là độ dài LCS của tiền tố $S[0 \dots i-1]$ và $T[0 \dots j-1]$.
- **Công thức chuyển trạng thái**:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{nếu } S[i-1] == T[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{nếu } S[i-1] \neq T[j-1] \end{cases}$$

- **Trường hợp cơ sở**: $dp[0][j] = 0, dp[i][0] = 0$.
- **Độ phức tạp**: $O(N \cdot M)$ thời gian và không gian.
- **Truy vết (Reconstruction)**: Bắt đầu từ (N, M) , đi ngược về $(0, 0)$:
 - Nếu $S[i-1] == T[j-1]$: Ký tự này thuộc LCS $\implies$ ghi nhận và lùi về $(i-1, j-1)$.
 - Nếu $dp[i-1][j] \geq dp[i][j-1]$: Lùi về $(i-1, j)$.
 - Ngược lại: Lùi về $(i, j-1)$.

3.4. Mô hình Quy hoạch Động trên Lưới 2D (Grid DP)

Cho lưới kích thước $N \times M$. Bạn xuất phát từ ô $(1, 1)$ và chỉ được phép di chuyển sang **phải** $(r, c + 1)$ hoặc xuống **dưới** $(r + 1, c)$ để tới ô (N, M) .

Bài toán 1: Đếm số đường đi duy nhất (Unique Paths)

- $dp[r][c] = (dp[r - 1][c] + dp[r][c - 1]) \pmod{10^9 + 7}$.
- Nếu ô (r, c) có vật cản: $dp[r][c] = 0$.

Bài toán 2: Đường đi có tổng chi phí nhỏ nhất (Min Path Sum)

Mỗi ô (r, c) có chi phí $cost[r][c]$.

- $dp[r][c] = cost[r][c] + \min(dp[r - 1][c], dp[r][c - 1])$.

Kỹ thuật Mảng Cuộn 1D cho Lưới:

Vì $dp[r][c]$ chỉ cần giá trị của ô phía trên $(dp[r - 1][c])$ và ô bên trái $(dp[r][c - 1])$, ta có thể rút gọn bảng $N \times M$ thành mảng 1D kích thước M :

```
vector<long long> dp(M, INF);
dp[0] = cost[0][0];
for (int r = 0; r < N; ++r) {
    for (int c = 0; c < M; ++c) {
        if (r == 0 && c == 0) continue;
        long long from_top = (r > 0 ? dp[c] : INF);
        long long from_left = (c > 0 ? dp[c - 1] : INF);
        dp[c] = cost[r][c] + min(from_top, from_left);
    }
}
```

3.5. Bài toán Đổi tiền: Phân biệt Hoán vị (Permutations) vs. Tổ hợp (Combinations)

Một trong những cạm bẫy kinh điển nhất trong CP khi làm bài toán Đổi tiền (Coin Change) lấy dư:

Bài toán A: Đếm số TỔ HỢP đổi tiền (Combinations - Không phân biệt thứ tự)

Ví dụ: Các đồng xu $\{2, 3, 5\}$ đổi số tiền 9. Cách $(2 + 2 + 5)$ và $(5 + 2 + 2)$ được xem là **CÙNG MỘT CÁCH**.

- **Quy tắc vòng lặp**: Vòng lặp duyệt từng đồng xu nằm **Ở NGOÀI**, vòng lặp số tiền nằm **Ở TRONG**:

```
// Vòng lặp COIN ở ngoài -> Chỉ tạo ra các dãy không giảm -> TỔ HỢP
dp[0] = 1;
for (int coin : coins) {
    for (int w = coin; w <= target; ++w) {
        dp[w] = (dp[w] + dp[w - coin]) % MOD;
    }
}
```

Bài toán B: Đếm số HOÁN VỊ đổi tiền (Permutations - Có phân biệt thứ tự)

Ví dụ: Cách $(2 + 2 + 5)$ và $(5 + 2 + 2)$ được tính là **HAI CÁCH KHÁC NHAU**.

- **Quy tắc vòng lặp**: Vòng lặp số tiền nằm **Ở NGOÀI**, vòng lặp duyệt từng đồng xu nằm **Ở TRONG**:

```
// Vòng lặp TIỀN ở ngoài -> Tại mỗi mức tiền thử mọi đồng xu -> HOÁN VỊ
dp[0] = 1;
for (int w = 1; w <= target; ++w) {
    for (int coin : coins) {
        if (w >= coin) {
            dp[w] = (dp[w] + dp[w - coin]) % MOD;
        }
    }
}
```

4. Trace Thuật toán Từng bước: 0/1 Knapsack

Cho balo sức chứa $W = 5$ và 3 đồ vật:

- Vật 1: $w_1 = 2, v_1 = 3$
- Vật 2: $w_2 = 3, v_2 = 4$
- Vật 3: $w_3 = 4, v_3 = 5$

Khởi tạo mảng 1D: $dp = [0, 0, 0, 0, 0, 0]$.

Bước	Đồ vật xét	Chiều duyệt j	Cập nhật $dp[j] = \max(dp[j], dp[j - w] + v)$	Trạng thái mảng $dp[0 \dots 5]$ sau bước
0	Khởi tạo	—	Không xét	$[0, 0, 0, 0, 0, 0]$
1	Vật 1 ($w = 2, v = 3$)	$5 \rightarrow 2$	$dp[5] = \max(0, dp[3] + 3) = 3$ $dp[4] = \max(0, dp[2] + 3) = 3$ $dp[3] = \max(0, dp[1] + 3) = 3$ $dp[2] = \max(0, dp[0] + 3) = 3$	$[0, 0, 3, 3, 3, 3]$
2	Vật 2 ($w = 3, v = 4$)	$5 \rightarrow 3$	$dp[5] = \max(3, dp[2] + 4) = \max(3, 3 + 4) = 7$ $dp[4] = \max(3, dp[1] + 4) = 4$ $dp[3] = \max(3, dp[0] + 4) = 4$	$[0, 0, 3, 4, 4, 7]$
3	Vật 3 ($w = 4, v = 5$)	$5 \rightarrow 4$	$dp[5] = \max(7, dp[1] + 5) = 7$ $dp[4] = \max(4, dp[0] + 5) = 5$	$[0, 0, 3, 4, 5, 7]$

Kết quả tối ưu: $dp[5] = 7$ (chọn Vật 1 và Vật 2: tổng khối lượng $2 + 3 = 5 \leq 5$, tổng giá trị $3 + 4 = 7$).

⚠ 5. Chạm bẫy Phòng thi & Kỹ thuật Tối ưu (Traps & Pitfalls)

1. **Tràn số 32-bit (`int` overflow):**
2. Các bài toán DP đếm số cách (như đếm đường đi, đếm cách đổi tiền) có kết quả tăng theo hàm mũ.
Luôn lấy modulo $10^9 + 7$ (hoặc 998244353) ở **mọi phép cộng**:

```
dp[w] = (dp[w] + dp[w - coin]) % MOD;
```
3. Với các bài toán tìm Min/Max tổng chi phí, tổng có thể vượt quá $2 \cdot 10^9$. Bắt buộc khai báo kiểu `long long`.
4. **Khởi tạo giá trị vô cùng (∞) an toàn:**
5. Khi tìm **min**, ta thường khởi tạo `dp` bằng một giá trị rất lớn.
6. Nếu dùng `LLONG_MAX` ($2^{63} - 1$), khi thực hiện phép cộng `dp[u] + cost` sẽ bị **tràn số thành số âm**, làm sai toàn bộ kết quả!
7. Giá trị an toàn chuẩn mực:
 - Với kiểu `int`: Dùng `1e9` hoặc `0x3f3f3f3f` (khoảng $1.06 \cdot 10^9$).
 - Với kiểu `long long`: Dùng `1e18` hoặc `2e18`.
8. **Thứ tự vòng lặp trong mảng nén 1D:**
9. 0/1 Knapsack: Bắt buộc duyệt ngược $W \rightarrow w_i$.
10. Unbounded Knapsack: Bắt buộc duyệt xuôi $w_i \rightarrow W$.
11. Nhầm lẫn giữa hai chiều duyệt này là lỗi phổ biến nhất dẫn đến WA trong các kỳ thi.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.